

Deep Learning - Projet

AKIBODE Yayrale

23 novembre 2025

Résumé

Ce projet vise à explorer le fonctionnement des réseaux de neurones en utilisant un dataset qui fait la reconnaissance d'activités humaines à partir de données de capteurs (HAR dataset). L'objectif n'est pas d'obtenir de meilleures performances, mais de tester différentes architectures et méthodes afin de comprendre l'apprentissage et le traitement des données. Nous avons étudié l'impact des fonctions d'activation, des optimisateurs, du nombre de neurones et des techniques de régularisation telles que le Dropout, le Weight Decay, la Batch Normalization et l'Early Stopping. Enfin, nous avons utilisé des métriques comme l'accuracy, le F1-score et la matrice de confusion pour analyser les performances et visualiser les représentations apprises par le réseau.

Introduction.

Depuis quelques décennies, le deep learning suscite beaucoup d'intérêt. Les réseaux de neurones sont souvent présentés comme capables de résoudre presque tous les problèmes, mais ils restent perçus comme très complexes et difficiles à comprendre.

Dans ce projet, nous cherchons à démystifier certaines propriétés de ces réseaux. Nous travaillerons avec un dataset réel de reconnaissance d'activités humaines à partir de données de capteurs (HAR dataset). La variable à prédire comporte 6 classes donc nous sommes dans le cadre d'un problème de classification multiclassées.

L'objectif n'est pas d'obtenir les meilleures prédictions possibles sur les données de test. Nous voulons plutôt expérimenter plusieurs architectures et différentes méthodes afin de mener une discussion critique sur le traitement et l'apprentissage dans les réseaux de neurones. Pour cela nous allons étudier l'incidence des fonctions d'activation, des optimisateurs, du nombre de neurones et des techniques de régularisation telles que le Dropout, le Weight Decay, la Batch Normalization et l'Early Stopping. Nous allons également tester quelques méthodes de réduction de dimension.

Pour comparer les différents modèles, nous allons utiliser des métriques comme l'accuracy, le F1-score et la matrice de confusion.

1 Description des données.

1.1 Statistiques descriptives

Notre dataset est splitté en deux jeux de données : train et test. Les informations ci-dessous n'englobent pas la variable cible 'Activity'.

Dataset	Nbre d'observations	Nbre de variables	Type de données
train	7352	562	Numérique
test 4	2947	562	Numérique

TABLE 1 – Informations sur le dataset.

Toutes les données sont numériques donc nous n'avons pas besoin d'effectuer un préprocessing avant l'apprentissage. D'autre part, sur l'image ci-dessous nous pouvons remarquer que les valeurs numériques varient entre -1 et 1. Les données semblent donc standardisées.

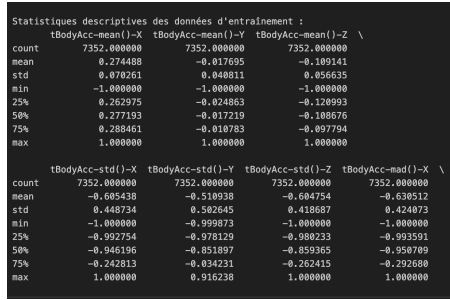


FIGURE 1 – Statistiques descriptives des premières variables.

1.2 Visualisation de la distribution de quelques variables

Les distributions des variables sont assez diversifiées. Nous pouvons déterminer des gaussiennes centrées, des gaussiennes non centrées, de mélanges de gaussiennes et des distributions asymétriques.

Nous avons également visualiser d'autres indicateurs comme la matrice de confusion (cf. 'version_1.ipynb'). À première vue, nous pouvons dire que la grande partie des variables ne sont pas très corrélées. Cependant, il y a quelques variables qui sont très positivement corrélées.

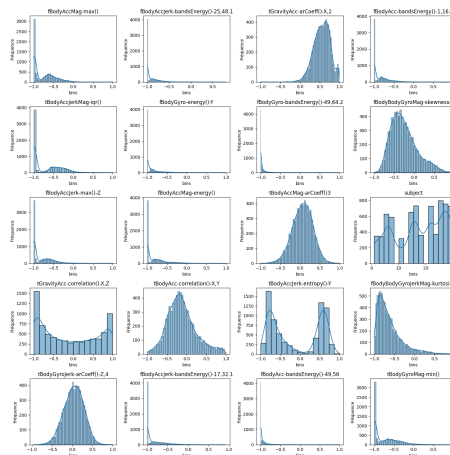


FIGURE 2 – Distribution de quelques variables.

1.3 Visualisation de la variables cible : 'Activity'

La variable à prédire 'Activity' comprend six classes. Dans sa visualisation nous pouvons remarquer que la classe 'WALKING_DOWNSTAIRS' est en minorité. D'autre part, la répartition générale

des observations dans les classes est légèrement asymétrique. Ces observations nous seront utiles plus tard dans le choix des mesures de performance.

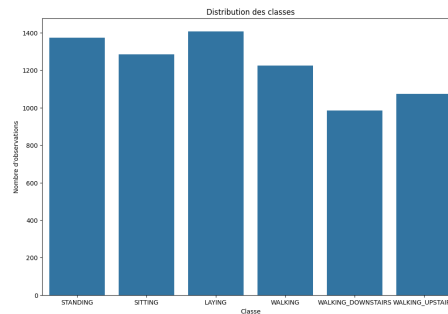


FIGURE 3 – Distribution de la variable 'Activity'.

2 Architectures

Maintenant que nous avons parcouru les données, nous allons passer aux architectures. Dans ce projet nous avons essentiellement travaillé avec les perceptrons multi-couches.

Dans une première partie, nous commençons par travailler avec une architecture avec une seule couche cachée. L'objectif de cette étude est de voir l'incidence du choix des fonctions d'activation, des algorithmes d'optimisation et du nombre de neurones sur la couche sur les performances du modèle.

Dans l'étape suivante, nous allons insérer plusieurs couches cachées afin de voir s'il y a des améliorations de performance. Nous allons également testé quelques bonnes pratiques telles la régularisation et la réduction de dimension afin d'étudier leur contribution à l'amélioration des performances.

2.1 Configuration de l'architecture avec une seule couche cachée

Pour l'apprentissage des poids nous allons utiliser des fonctions déjà vues au cours. Cela nous permet d'avoir la main sur le processing et d'y ajouter des fonctionnalités.

La configuration est comme suit :

Listing 1 – Configuration des hyperparamètres

```
CONFIG = {
    'BATCH_SIZE': 32,
    'EPOCHS': 100,
    'LR': 1e-3,
    'hidden_dims': [64, 128, 256],
    'optimizer': ['Adam', 'SGD', 'SGD_Momentum'],
    'activation': ['ReLU', 'Sigmoid', 'Tanh']
}
```

Pour analyser un paramètre en particulier, nous allons fixer les autres. Dans notre étude nous allons généralement fixer 64 comme nombre de neurones, 'Adam' comme algorithme d'optimisation et 'Relu' comme fonction d'activation.

En ce qui concerne le critère, nous allons utiliser la cross-entropie pour tous les modèles. De fait, c'est le critère qui correspond à un problème de classification multiclassées.

2.2 Configuration de l'architecture avec plusieurs couches cachées

Nous augmentons maintenant, le nombre de couches cachées. Toujours dans ce même sens nous allons maintenant essayer quelques types de régularisation.

- **Dropout** : désactive aléatoirement un pourcentage de neurones pendant l'entraînement, ce qui empêche le réseau de trop dépendre de certains neurones et réduit le sur-apprentissage.
- **Weight Decay (L2)** : ajoute une pénalité sur la taille des poids du modèle afin de les limiter, ce qui améliore la capacité de généralisation.
- **Batch Normalization** : normalise les activations dans chaque couche pour stabiliser et accélérer l'entraînement tout en limitant l'overfitting.

- **Early Stopping** : arrête automatiquement l'entraînement si les performances sur la validation ne s'améliorent plus, afin d'éviter le sur-apprentissage.

Listing 2 – Configuration des hyperparamètres

```
CONFIG = {
    'BATCH_SIZE': 32,
    'EPOCHS': 100,
    'LR': 1e-3,
    'hidden_dims': [[128, 64], [256, 128, 64], [512, 256, 128, 64]]
    'optimizer': ['Adam', 'SGD', 'SGD_Momentum'],
    'activation': ['ReLU', 'Sigmoid', 'Tanh'],
    'p_dropout' : [0.1, 0.3, 0.5]
    'weight_decay' : [1e-2, 1e-3, 1e-4]
}
```

3 Analyses

3.1 Mesures de performance

Pour pouvoir analyser les résultats et tirer des conclusions, nous avons besoin de mesures. En classification, nous avons l'Accuracy, la F1-score, la courbe d'apprentissage et la matrice de confusion. Dans nos analyses, nous allons utiliser les quatres car elles nous procurent des informations différentes.

- **Accuracy** : proportion de prédictions correctes parmi l'ensemble des prédictions. Une bonne métrique si les classes sont équilibrées.
- **F1-score** : moyenne harmonique de la précision et du rappel. Permet d'évaluer correctement les performances même en cas de classes déséquilibrées. Le *F1-score macro* donne un score moyen sur toutes les classes.
- **Courbe d'apprentissage** : représentation de l'évolution des performances (perte, accuracy ou F1) en fonction des époques d'entraînement. Permet d'identifier l'underfitting, l'overfitting et la bonne convergence du modèle.
- **Matrice de confusion** : tableau comparant pour chaque classe le nombre de prédictions correctes et incorrectes. Permet de visualiser quelles classes sont bien ou mal reconnues par le modèle.

3.2 Analyse : Modèle avec une seule couche cachée

La configuration des différents modèles est résumée dans le tableau ci-dessous.

Modèle	Activation	Optimiseur	Neurones (couche cachée)
Modèle 5.1	ReLU	Adam	64
Modèle 5.2	Sigmoid	Adam	64
Modèle 5.3	Tanh	Adam	64
Modèle 5.4	ReLU	SGD	64
Modèle 5.5	ReLU	SGD Momentum	64
Modèle 7.1	ReLU	Adam	128
Modèle 7.2	ReLU	Adam	256

TABLE 2 – Configurations des différents modèles testés.

3.2.1 Courbe d'apprentissage et Accuracy

La test loss présentent un overfitting, sauf pour les modèles avec un algorithme SGD. Cependant en terme de stabilité, la SGD simple est très stable comparativement à la SGD avec momentum.

En ce qui concerne les fonctions d'activation, il n'y a pas une grande incidence sur les performances. Cependant, nous observons un changement de monotonie avec la fonction d'activation Tanh.

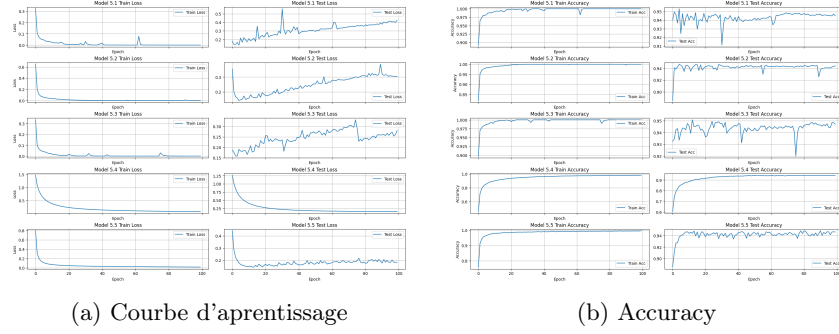


FIGURE 4 – Courbe d'apprentissage et Accuracy

3.2.2 F1-score et Matrices de confusion

Ces mesures nous donnent plus d'informations sur la prédiction de chaque classe. Il y a une mauvaise prédiction des classes STANDING et SITTING. Cela est particulièrement visible sur les boxplots des f1 par classe.

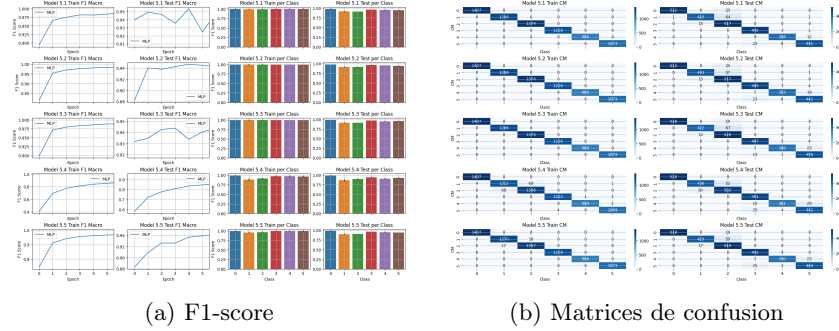


FIGURE 5 – F1-score et Matrices de confusion

3.2.3 Augmentation du nombre de neurones

L'augmentation du nombre de neurones n'apportent pas une grande amélioration de la performance.

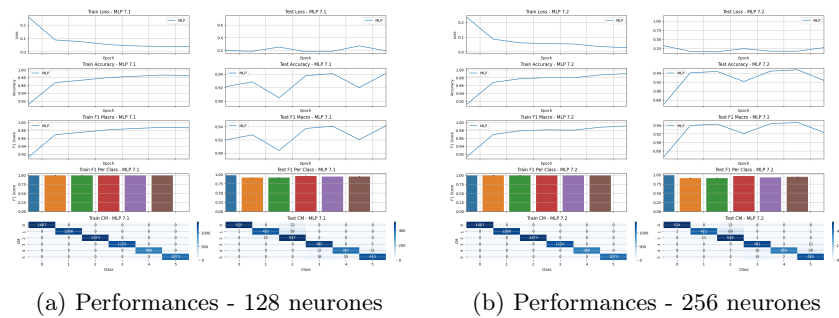


FIGURE 6 – Performances - 128 et 256 neurones

3.3 Analyse : Modèle avec plusieurs couches cachées

La configuration des différents modèles est résumée dans le tableau ci-dessous.

Modèle	Activation	Optimiseur	Neurones (dimension des couches cachées)
Modèle 5.1	ReLU	Adam	[128, 64]
Modèle 5.2	Sigmoid	Adam	[128, 64]
Modèle 5.3	Tanh	Adam	[128, 64]
Modèle 5.4	ReLU	SGD	[128, 64]
Modèle 5.5	ReLU	SGD Momentum	[128, 64]
Modèle 7.1	ReLU	Adam	[256, 128, 64]
Modèle 7.2	ReLU	Adam	[512, 256, 128, 64]

TABLE 3 – Configurations des différents modèles testés.

3.3.1 Courbe d'apprentissage et Accuracy

Avec l'augmentation de couches cachées, l'overfitting est beaucoup plus contrôlé. Cependant, l'algorithme SGD reste le plus stable.

Nous avons également augmenté l'ordre de grandeur de l'accuracy. Dans la sous-section précédente cette dernière était de l'ordre de 0.94. Maintenant, elle est de l'ordre de 0.95

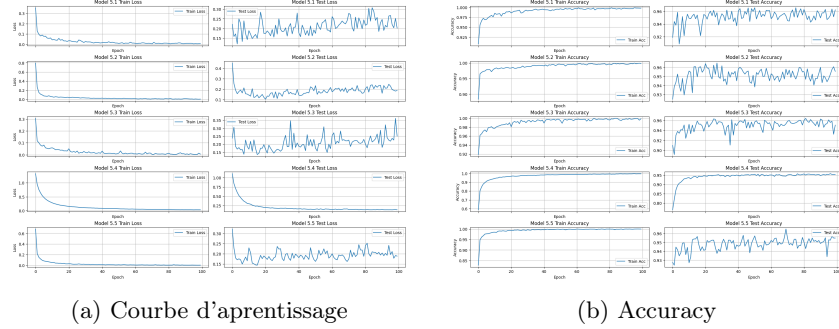


FIGURE 7 – Courbe d'apprentissage et Accuracy

3.3.2 F1-score et Matrice de confusion

La prédiction des classes STANDING et SITTING ne s'est toujours pas améliorée.

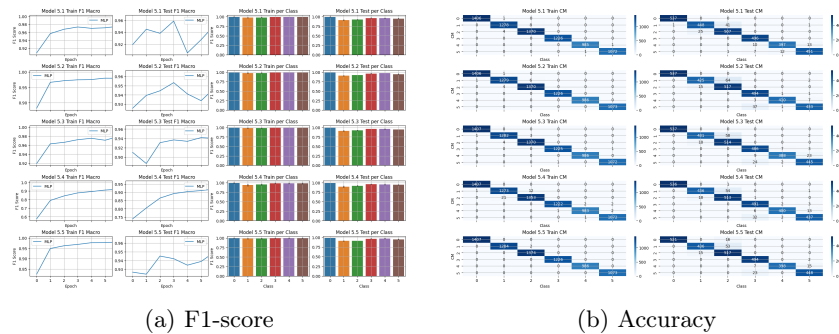


FIGURE 8 – Matrice de confusion

3.3.3 Analyse : Augmentation du nombre de couches

L'augmentation du nombre de couches n'apportent pas une grande amélioration de la performance.

3.3.4 Analyse : Régularisation

- **Dropout** : Pour $p_{dropout} = 0.5$, nous atteignons une accuracy de 0.96 et l'overfitting est très bien contrôlé.

- **Weight Decay (L2)** : Il n'apporte pas une grande amélioration si ce n'est une légère amélioration de la prédiction des classes 'STANDING' et 'SITTING'.
- **Batch Normalization** : contrôle assez bien l'overfitting.
- **Early Stopping** : s'il fait gagner en temps, cela a augmenté les mauvaises prédictions de la classe 'STANDING'.

3.3.5 Réduction de dimension : t-SNE et UMAP

La réduction de dimension permet de projeter des données de haute dimension dans un espace plus faible, souvent pour visualiser les représentations internes d'un modèle.

- **t-SNE** : préserve les relations locales entre points et met en évidence des regroupements. Très coûteux en calcul pour de grands ensembles de données.
- **UMAP** : modélise la structure locale et globale, plus rapide que t-SNE et souvent plus stable.

L'objectif était d'appliquer ces méthodes aux activations de la dernière couche cachée pour visualiser comment le modèle distingue les classes. Cependant, leur coût computationnel étant trop élevé, nous n'avons pas pu les tester dans ce projet.

3.3.6 Ouverture : LayerNorm

Elle normalise les activations d'une couche pour chaque échantillon, ce qui stabilise et accélère l'entraînement du réseau, tout en aidant à réduire le sur-apprentissage.

Dans notre cas précis, elle n'apporte pas une grande amélioration. L'accuracy est de l'ordre de 0.94 et il y a de l'overfitting.

4 Discussion critique

- Lorsque nous avons visualiser la distribution de la variable cible 'Activity', nous avons remarqué que la classe 'WALKING_DOWNSTAIRS' était en minorité. Il était intuitif de penser que ce serait la classe la plus mal prédite. Cependant, ça n'a pas été le cas. De fait, c'étaient les classes 'STANDING' et 'SITTING' qui étaient mal prédites. Nous pouvons expliquer cela par le fait que les mesures physiologiques d'un être humain sont assez stables lorsqu'on est debout ou assis. Mais lorsqu'on marche, il y a une instabilité donc plus facile à détecter.
- D'autre part, nous avons remarqué que les fonctions d'activation n'ont pas une grande incidence sur le modèle. L'algorithme SGD pour sa part s'est montré très performant dans le contrôle de l'overfitting. En effet, cela est dû à la pseudo-standardisation des données. Car cet algorithme explose sur des données de grande échelle.
- Par la suite, lorsque nous sommes passés à une architecture avec 2 couches cachées. Les performances se sont améliorées. De même l'overfitting était bien contrôlé. Cependant, l'augmentation du nombre de neurones sur une couche ou l'augmentation du nombre de couches cachés n'améliorent pas forcément les performances comme nous le pensons souvent.
- Enfin, les différentes méthodes de régularisation ne nous ont pas apporté une grande amélioration. Cependant, nous les avons testé de façon indépendante. Il se pourrait que la combinaison de plusieurs méthodes nous donnent de très bonnes performances.

5 Conclusion

Ce projet nous a permis d'explorer le fonctionnement des réseaux de neurones en étudiant plusieurs combinaisons de configuration. Nos expérimentations montrent que certaines intuitions ne se vérifient pas toujours : par exemple, les classes minoritaires ne sont pas forcément les plus mal prédites, et l'augmentation du nombre de neurones ou de couches ne garantit pas une amélioration des performances. Les techniques de régularisation ont un impact limité lorsqu'elles sont utilisées isolément, mais pourraient être plus efficaces combinées. Enfin, le choix de l'optimiseur et la standardisation des données restent cruciaux pour contrôler le sur-apprentissage. Ce travail souligne l'importance d'une analyse critique et expérimentale pour mieux comprendre le comportement des réseaux de neurones sur des données réelles.